UNIT V

Advanced Topics:

1. Explain Minimax search procedure - Adding alpha-beta cutoffs in Game Playing

Adversarial Search

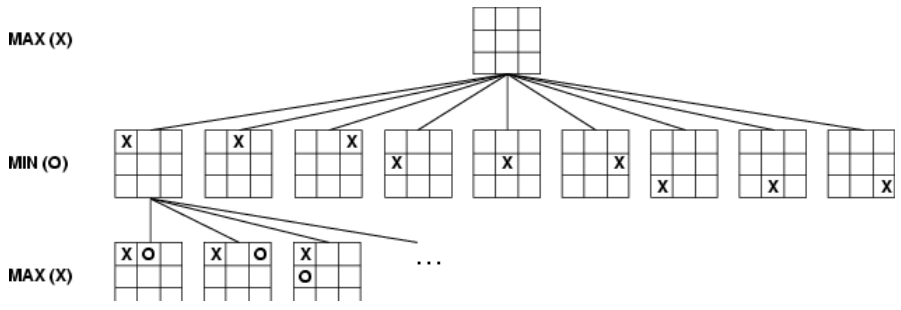
Competitive environments, in which the agents' goals are in conflict, give rise to adversarial search problems- often known as games. In AI, "games" are usually of a rather specialized kind – in which there are two agents whose actions must alternate and in which the utility values at the end of the game are always equal and opposite.

A game can be formally defined as a kind of search problem with the following components:

- The **initial state**, which includes the board position and identifies the player to move.
- A **successor function**, which returns a list of (move, state)pairs, each indicating a legal move and the resulting state.
- A **terminal test**, which determines when the game is over. States where the game has ended are called terminal states.
- A **utility function**, which gives a numeric value for the terminal states.

The initial state and the legal moves for each side define the **game tree** for the game. The following figure shows part of the game tree for tic-tac-toe. From the initial state, MAX has nine possible moves. Play alternates between MAX's placing an X and MIN's placing an O until we reach leaf nodes corresponding to terminal states such that one player has three in a row or all the squares are filled.

Game tree (2-player, deterministic, turns)

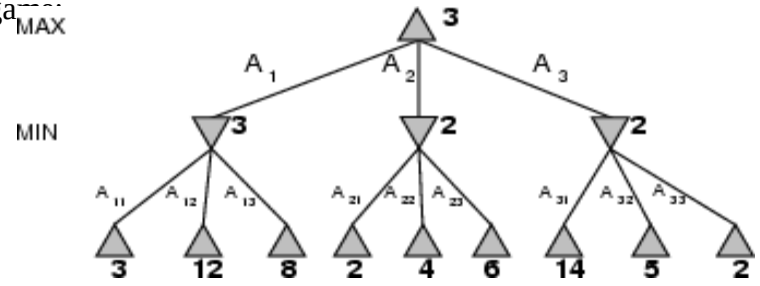


MINIMAX

Given a game tree, the optimal strategy can be determined by examining the minimax value of each node, which we write as MINIMAX-VALUE(n). The minimax value of a node is the utility of being in the corresponding state, assuming that both players play optimally from there to the end of the game.

- Perfect play for deterministic games
- Idea: choose move to position with highest minimax value

  = best achievable payoff against best play
- E.g., 2-ply game:



Here, the first MIN node, labeled B, has three successors with values 3,12, and 8, so its minimax value is 3. The minimax algorithm computes the minimax decision from the current state.

Minimax algorithm

```
function MINIMAX-DECISION(state) returns an action
    v ← MAX-VALUE(state)
    return the action in SUCCESSORS(state) with value v

function MAX-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s))
    return v
```

Properties of minimax

- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$:the maximum depth of the tree is m, and there are b legal moves at each point
    - **Space complexity? O(bm) (depth-first**
- With "perfect ordering," time complexity = $O(b^{m/2})$
  → doubles depth of search
- exploration)
- For chess, b ≈35, m ≈100 for "reasonable" games
  → exact solution completely infeasible

ALPHA-BETA PRUNING

The problem with minimax procedure is that the number of game states it has to examine is exponential in the number of moves. We can cut it in half using the technique called alpha-beta pruning. When applied to a standard minimax tree, it returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.

Consider again the two-ply game tree. The steps are explained in the following figure. The outcome is that we can identify the minimax decision without ever evaluating two of the leaf nodes.

α-β pruning example

The value of the root node is given by

MINIMAX-VALUE(root) = max(min(3,12,8), min(2,x,y), min(14,5,2))

= max(3,min(2,x,y),2)

=max(3,z,2) where z<=2

=3

x and y: two unevaluated successors

z: minimum of x and y

Properties of α-β

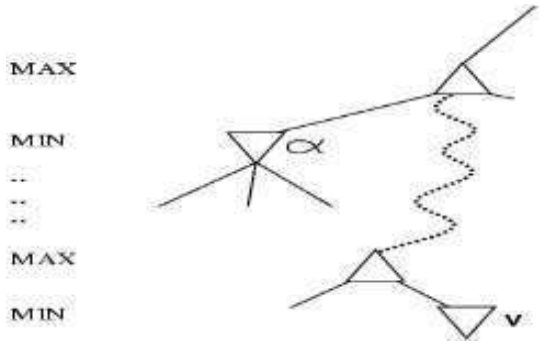
- Pruning does not affect final result

- Good move ordering improves effectiveness of pruning
- A simple example of the value of reasoning about which computations are relevant (a form of metareasoning)

Why is it called α-β?

- α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for *max*
- β is the value of the best (i.e., lowest-value) choice found so far at any choice point along the path for *min*
- If *v* is worse than α, *max* will avoid it

  → prune that branch



Alpha-beta search updates the values of α and β as it goes along and prunes the remaining branches at a node as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN respectively. The effectiveness of alpha-beta pruning is highly dependent on the order in which the successors are examined. The algorithm is given below:

The α-β algorithm

```
function ALPHA-BETA-SEARCH(state) returns an action
   inputs: state, current state in game
   v ← MAX-VALUE(state, −∞, +∞)
   return the action in SUCCESSORS(state) with value v

function MAX-VALUE(state, α, β) returns a utility value
   inputs: state, current state in game
           α, the value of the best alternative for MAX along the path to state
           β, the value of the best alternative for MIN along the path to state
   if TERMINAL-TEST(state) then return UTILITY(state)
   v ← −∞
   for a, s in SUCCESSORS(state) do
      v ← MAX(v, MIN-VALUE(s, α, β))
      if v ≥ β then return v
      α ← MAX(α, v)
   return v
```

```
function MIN-VALUE(state, α, β) returns a utility value
   inputs: state, current state in game
           α, the value of the best alternative for MAX along the path to state
           β, the value of the best alternative for MIN along the path to state
```

2. Explain Expert System its Expert System shells and Knowledge Acquisition.

Expert Systems (ES), also called Knowledge-Based Systems (KBS) or simply Knowledge Systems (KS), are computer programs that use expertise to assist people in performing a wide variety of functions, including diagnosis, planning, scheduling and design. ESs are distinguished from conventional computer programs in two essential ways (Barr, Cohen et al. 1989):

1. Expert systems reason with domain-specific knowledge that is symbolic as well as numerical;
2. Expert systems use domain-specific methods that are heuristic (i.e., plausible) as well as algorithmic

The technology of expert systems has had a far greater impact than even the expert systems business. Expert system technology has become widespread and deeply embedded. As expert system techniques have matured into a standard information technology, the most important recent trend is the increasing integration of this technology with conventional information processing, such as data processing or management information systems.

The Building Blocks of Expert Systems

Every expert system consists of two principal parts: the knowledge base; and the reasoning, or inference, engine.

The *knowledge base* of expert systems contains both factual and heuristic knowledge. *Factual knowledge* is that knowledge of the task domain that is widely shared, typically

found in textbooks or journals, and commonly agreed upon by those knowledgeable in the particular field.

Heuristic knowledge is the less rigorous, more experiential, more judgmental knowledge of performance. In contrast to factual knowledge, heuristic knowledge is rarely discussed, and is largely individualistic. It is the knowledge of good practice, good judgment, and plausible reasoning in the field. It is the knowledge that underlies the "art of good guessing."

Knowledge representation formalizes and organizes the knowledge. One widely used representation is the *production rule*, or simply *rule*. A rule consists of an IF part and a THEN part (also called a *condition* and an *action*). The IF part lists a set of conditions in some logical combination. The piece of knowledge represented by the production rule is relevant to the line of reasoning being developed if the IF part of the rule is satisfied; consequently, the THEN part can be concluded, or its problem-solving action taken. Expert systems whose knowledge is represented in rule form are called *rule-based systems*.

Another widely used representation, called the *unit* (also known as *frame, schema*, or *list structure*) is based upon a more passive view of knowledge. The unit is an assemblage of associated symbolic knowledge about an entity to be represented. Typically, a unit consists of a list of properties of the entity and associated values for those properties.

Since every task domain consists of many entities that stand in various relations, the properties can also be used to specify relations, and the values of these properties are the names of other units that are linked according to the relations. One unit can also represent knowledge that is a "special case" of another unit, or some units can be "parts of" another unit.

The *problem-solving model*, or *paradigm*, organizes and controls the steps taken to solve the problem. One common but powerful paradigm involves chaining of IF-THEN rules to form a line of reasoning. If the chaining starts from a set of conditions and moves toward some conclusion, the method is called *forward chaining*. If the conclusion is known (for example, a goal to be achieved) but the path to that conclusion is not known, then reasoning backwards is called for, and the method is *backward chaining*. These problem-solving methods are built into program modules called *inference engines* or *inference procedures* that manipulate and use knowledge in the knowledge base to form a line of reasoning.

The most important ingredient in any expert system is knowledge. The power of expert systems resides in the specific, high-quality knowledge they contain about task domains. AI researchers will continue to explore and add to the current repertoire of knowledge representation and reasoning methods. But in knowledge resides the power. Because of the importance of knowledge in expert systems and because the current knowledge acquisition method is slow and tedious, much of the future of expert systems depends on breaking the knowledge acquisition bottleneck and in codifying and representing a large knowledge infrastructure.

expert system shell

A rule-based, expert system maintains a separation between its Knowledge-base and that part of the system that executes rules, often referred to as the ***expert system shell***. The systemshell is indifferent to the rules it executes. This is an important distinction, because it means that the expert system shell can be applied to many different problem domains with little or no change. It also means that adding or modifying rules to an expert system can effect changes in program behavior without affecting the controlling component, the system shell.

The language used to express a rule is closely related to the language ***subject matter experts*** use to describe problem solutions. When the subject matter expert composes a rule using this language, he is, at the same time, creating a written record of problem knowledge, which can then be shared with others. Thus, the creation of a rule kills two birds with one stone; the rule adds functionality or changes program behavior, and records essential information about the problem domain in a human-readable form. Knowledge captured and maintained by these systems ensures continuity of operations even as subject matter experts (i.e., mathematicians, accountants, physicians) retire or transfer.

Furthermore, changes to the Knowledge-base can be made easily by subject matter experts without programmer intervention, thereby reducing the cost of software maintenance and helping to ensure that changes are made in the way they were intended. Rules are added to the knowledge-base by subject matter experts using text or graphical editors that are integral to the system shell. The simple process by which rules are added to the knowledge-base is depicted in Figure 1.

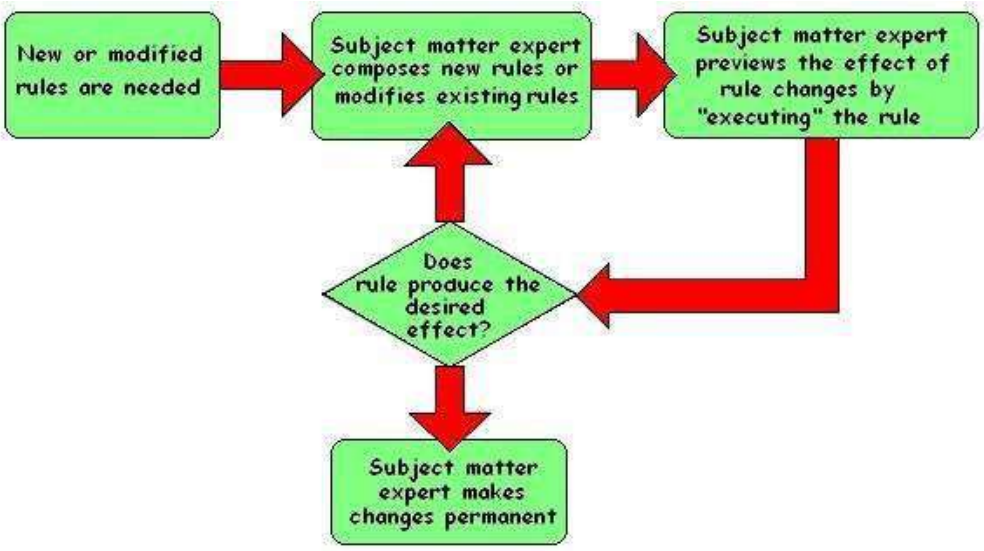


Figure 1 - Creating a knowledge-base

Finally, the expert system never forgets, can store and retrieve more knowledge than any single human being can remember, and makes no errors, provided the rules created by the subject matter experts accurately model the problem at hand.

4. EXPERT SYSTEM ARCHITECTURE

An expert system is, typically, composed of two major components, the Knowledge-base and the Expert System Shell. The Knowledge-base is a collection of rules encoded as ***metadata*** in a file system, or more often in a relational database. The Expert System Shell is a problem-independent component housing facilities for creating, editing, and executing rules. A software architecture for an expert system is illustrated in Figure 2.

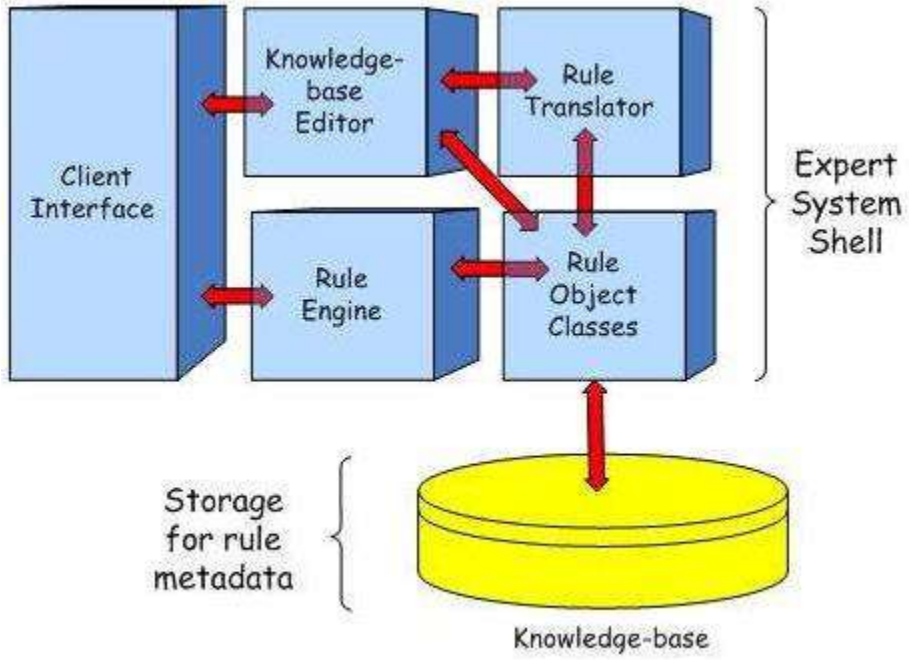


Figure 2 - Expert System Architecture

The **shell** portion includes software modules whose purpose it is to,

- Process requests for service from system users and application layer modules;
- Support the creation and modification of business rules by subject matter experts;
- Translate business rules, created by a subject matter experts, into machine-readable forms;
- Execute business rules; and
- Provide low-level support to expert system components (e.g., retrieve metadata from and save metadata to knowledge base, build Abstract Syntax Trees during rule translation of business rules, etc.).

3. Explain Robotics and its hardware,perception

Definition

"A reprogrammable, multifunctional manipulator designed to move material, parts, tools, or specialized devices through various programmed motions for the performance of a variety of tasks"

A robot has these essential characteristics:

- **Sensing** First of all your robot would have to be able to sense its surroundings. It would do this in ways that are not unsimilar to the way that you sense your surroundings. Giving your robot sensors: light sensors (eyes), touch and pressure sensors (hands), chemical sensors (nose), hearing and sonar sensors (ears), and taste sensors (tongue) will give your robot awareness of its environment.
- **Movement** A robot needs to be able to move around its environment. Whether rolling on wheels, walking on legs or propelling by thrusters a robot needs to be able to move. To count as a robot either the whole robot moves, like the Sojourner or just parts of the robot moves, like the Canada Arm.
- **Energy** A robot needs to be able to power itself. A robot might be solar powered, electrically powered, battery powered. The way your robot gets its energy will depend on what your robot needs to do.
- **Intelligence** A robot needs some kind of "smarts." This is where programming enters the pictures. A programmer is the person who gives the robot its 'smarts.' The robot will have to have some way to receive the program so that it knows what it is to do.

Robot Hardware

Sensors

Sensors are the perceptural interface between robots and their environments. Passive sensors. Such as cameras are true observers of the environment: they capture signals that are generated by other sources in the environment. Active sensors, such as sonar, send energy into the environment.

Many mobile robots make use of range finders, which are sensors that measure the distance to nearby objects. One common type is the sonar sensor, also known as and ultrasonic transducer.

Some range sensors measure very short or very long distances. Close-range sensors include tactile sensors such as whiskers,bumb panels, and touch-sensitive skin. At the other end of the spectrum is the Global Positioning System (GPS), which measures the distance to satellites that emit pulsed signals.

Effectors

Effectors are the means by which robots move and change the shape of their bodies. To understand the design of effectors,, it will help to talk about motion and shape in the abstract, using the concept of a degree of freedom(DOF).

The dynamic state of a robot includes one additional dimension for the rate of change of each kinematic dimension.

The arm in Figure 25.3(a) has exactly six degrees of freedom. Created by five revolute joints that generate rotational motion and one prismatic joint that generates sliding motion.

The car has 3 effective degrees of freedom but 2 controllable degress of freedom. We say a robot is nonholonomic if it has more effective DOF s than controllable DOFs and holonomic if the two numbers are the same.

For mobile robots, there exists a range of mechanisms for locomotion, including wheels,tracks and legs. Differential drive robots possess two independently actuated wheels.

ROBOTIC PERCEPTION

Perception is the process by which robots map sensor measurements into internal representations of the environment.Perception is difficult because in general the sensors are noisy, and the environment is partially observable, unpredictable, and often dynamic. As a rule of thumb, good internal representations have three properties: they contain enough information for the robot to make the right decisions, the are structured such that can be updated efficiently, and they are natural in the sense that internal varialble correspond to natural state variables in the physical world.

LOCALIZATION

Localization is a generic example of robot perception. It is the problem of determining where things are. Localization is on of the most pervasive perception problems in robotics,because knowledge about where things are is at the core of any successful physical insteraction.

The localization problem comes in three flavors of increasing difficulty. If the initial pose of the object to be localized by is known, localization is a tracking problem.Tracking problems are characterized by bounded uncertainly. More difficult is the global localization.

PLANNING TO MOVE

In robotics, decisions ultimately involve motion of effectors. The point–to –point motion problem is to deliver the robot or its end-effector to a designated target location. A greater challenge is the compliant motion problem, in which a robot moves while being in physical Contact with an obstacle. An example of compliant motion is a robot manipulator that screws in a light bulb, or a robot that pushes a box across a table top.

4. Explain Swarm Intelligent Systems .(0r) What is Ant Colony System, its Application and Working.

Swarm intelligence research originates from work into the simulation of the emergence of collective intelligent behaviors of real ants. Ants are able to find good solutions to the shortest path problems between the nest and a food source by laying down, on their way back from the food source, a trail of an attracting substance—a pheromone. Based on the pheromone level communication,the shortest path is considered that with the greatest density of pheromone and the ants will tend to follow the path with more pheromone. Dorigo and his colleagues were the first to apply this idea to the traveling salesman problem [1,2]. This algorithm is referred to as ant system (AS) algorithm

Ant system and ant colony system

Inspired by the food-seeking behavior of real ants, the ant system [1,2] is a cooperative population-based search algorithm. As each ant construct a route from nest to food by stochastically following the quantities of pheromone level,the intensity of laying pheromone will bias the path-choosing decision-make of subsequent ants.

The operation of ant system can be illustrated by the classical traveling salesman problem. A traveling salesman problem is seeking for a round route 64 S.-C. Chu et al. / Information Sciences 167 (2004) 63–76 covering all cities with minimal total distance. Suppose there are n cities and m ants.

The entire algorithm is started with initial pheromone intensity set to s0 on all edges. In every subsequent ant system cycle, or called episode, each ant begins its tour from a randomly selected starting city and is required to visit every city once and only once. The experience gained in this phase is then used to update the pheromone intensity on all edges.

The algorithm of the ant system for the traveling salesman problem (TSP) is depicted as follows [2,3]:

Step 1: Randomly select the initial city for each ant. The initial pheromone
level between any two cities is set to be a small positive constant.
Set the cycle counter to be 0.

Step 2: Calculate the transition probability from city r to city s for the kth ant
As

where r is the current city, s is the next city, sðr; sÞ is the pheromone level between city r and city s, gðr; sÞ ¼ 1=dðr; sÞ the inverse of the distance dðr; sÞ between city r and city s, JkðrÞ is the set of cities that remain to be visited by the kth ant positioned on city r, and b is a parameter which determines the relative importance of pheromone level versus distance. Select the next visited city s for the kth ant with the probability Pkðr; sÞ. Repeat Step 2 for each ant until the ants have toured all cities.
Step 3: Update the pheromone level between cities as

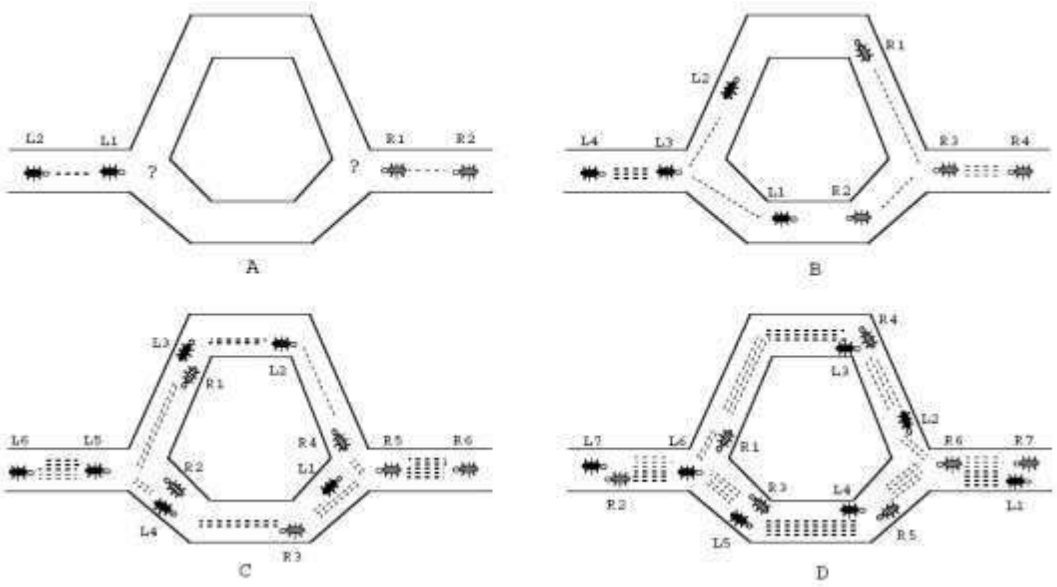
$$\tau(r,s) \leftarrow (1-\alpha)\cdot\tau(r,s) + \sum_{k=1}^{m}\Delta\tau_k(r,s),$$
$$\Delta\tau_k(r,s) = \begin{cases} \frac{1}{L_k} & \text{if } (r,s)\in \text{route done by ant } k, \\ 0 & \text{otherwise,} \end{cases}$$

Dskðr; sÞ is the pheromone level laid down between cities r and s by the kth ant, Lk is the length of the route visited by the kth ant, m is the number of ants and $0 < a < 1$ is a pheromone decay parameter.
Step 4: Increment cycle counter. Move the ants to the originally selected cities and continue Steps 2–4 until the behavior stagnates or the maximum number of cycles has reached, where a stagnation is indicated when all ants take the same route.

From Eq. (1) it is clear ant system (AS) needs a high level of computation to find the next visited city for each ant. In order to improve the search efficiency, the ant colony system (ACS) was proposed [3]. ACS is based on AS but updates the pheromone level before moving to the next city (local updating rule) and updating the pheromone level for the shortest route only after completing the route for each ant (global updating rule) as

$$\tau(r,s) \leftarrow (1-\alpha)\cdot\tau(r,s) + \alpha\cdot\Delta\tau(r,s),$$
$$\Delta\tau(r,s) = \begin{cases} (L_{\text{gb}})^{-1} & \text{if } (r,s)\in \text{global best route,} \\ 0 & \text{otherwise,} \end{cases}$$

where Lgb is the length of the shortest route and a is a pheromone decay
parameter



ACS differs from the previous ant system because of three main aspects: (i) the state transition rule provides a direct way to balance between exploration of new edges and exploitation of *a priori* and accumulated knowledge about the problem, (ii) the global updating rule is applied only to edges which belong to the best ant tour, and (iii) while ants construct a solution a *local pheromone updating rule* (local updating rule, for short) is applied.

Informally, ACS works as follows: *m* ants are initially positioned on *n* cities chosen according to some initialization rule (e.g., randomly). Each ant builds a tour (i.e., a feasible solution to the TSP) by repeatedly applying a stochastic greedy rule (the state transition rule). While constructing its tour, an ant also modifies the amount of pheromone on the visited edges by applying the local updating rule. Once all ants have terminated their tour, the amount of pheromone on edges is modified again (by applying the global updating rule). As was the case in ant system, ants are guided, in building their tours, by both heuristic information (they prefer to choose short edges), and by pheromone information: An edge with a high amount of pheromone is a very desirable choice. The pheromone updating rules are designed so that they tend to give more pheromone to edges which should be visited by ants. The ACS algorithm is reported in Fig. 3. In the following we discuss the state transition rule, the global updating rule, and the local updating rule.

A study of some characteristics of ACS

A. Pheromone behavior and its relation to performance

To try to understand which mechanism ACS uses to direct the search we study how the

pheromone–closeness product $[\tau(r, s)]\cdot[\eta(r, s)]\beta$ changes at run time. Fig. 4 shows how the pheromone–closeness product changes with the number of steps while ants are building a solution2 (steps refer to the inner loop in Fig. 3: the abscissa goes therefore from 1 to *n*, where *n* is the number of cities).

Let us consider three families of edges (see Fig. 4): (i) those belonging to the last best tour (BE, Best Edges), (ii) those which do not belong to the last best tour, but which did in one of the two preceding iterations (TE, Testable Edges), (iii) the remaining edges, that is, those that have never belonged to a best tour or have not in the last two iterations (UE, Uninteresting Edges).

The average pheromone–closeness product is then computed as the average of pheromone–closeness values of all the edges within a family. The graph clearly shows that ACS favors exploitation of edges in BE (BE edges are chosen with probability $q0=0.9$) and exploration of edges in TE (recall that, since Eqs. (3) and (1), edges with higher pheromone–closeness product have a higher probability of being explored).

An interesting aspect is that while edges are visited by ants, the application of the local updating rule, Eq. (5), makes their pheromone diminish, making them less and less attractive, and therefore favoring the exploration of edges not yet visited. Local updating has the effect of lowering the pheromone on visited edges so that these become less desirable and therefore will be chosen with a lower probability by the other ants in the remaining steps of an iteration of the algorithm.

As a consequence, ants never converge to a common path. This fact, which was observed experimentally, is a desirable property given that if ants explore different paths then there is a higher probability that one of them will find an improving solution than there is in the case that they all converge to the same tour (which would make the use of *m* ants pointless).

Experimental observation has shown that edges in BE, when ACS achieves a good performance, will be approximately downgraded to TE after an iteration of the algorithm (i.e., one external loop in Fig. 3; see also Fig. 4), and that edges in TE will soon be downgraded to UE, unless they happen to belong to a new shortest tour.

The optimal number of ants

Consider Fig. 63. Let $\phi 2\tau 0$ be the average pheromone level on edges in BE just after they are updated by the global updating rule, and $\phi 1\tau 0$ the average pheromone level on edges in BE just before they are updated by the global updating rule ($\phi 1\tau 0$ is also approximately the

average pheromone level on edges in TE at the beginning of the inner loop of the algorithm). Under the hypothesis that the optimal values of $\phi 1$ and $\phi 2$ are known, an estimate of the optimal number of ants can be computed as follows. The local updating rule is a first-order linear recurrence relation of the form $T\ T\ z\ z = - + -1\ 0\ (1\ \rho)\ \tau\ \rho$, which has closed form given by

$T\ T\ z= - z- - z+ 0\ 0\ 0\ (1\ \rho)\ \tau(1\ \rho)\ \tau$. Knowing that just before global updating $T0 = \phi 2\ \tau 0$ (this corresponds to the start point of the BE curve in Fig. 6), and that after all ants have built their tour and just before global updating, $Tz = \phi 1 \tau 0$

Cooperation among ants

This section presents the results of two simple experiments which show that ACS effectively exploits pheromone-mediated cooperation. Since artificial ants cooperate by exchanging information via pheromone, to have noncooperating ants it is enough to make ants blind to pheromone. In practice this is obtained by deactivating Eqs. (4) and (5), and setting the initial level of pheromone to $\tau 0=1$ on all edges.

When comparing a colony of cooperating ants witha colony of noncooperating ants, to make the comparison fair, we use CPU time to compute performance indexes so as to discount for the higher complexity, due to pheromone updating, of the cooperative approach.

In the first experiment, the distribution of *first finishing times*, defined as the time elapsed

until the first optimal solution is found, is used to compare the cooperative and the noncooperative approaches. The algorithm is run 10,000 times, and then we report on a graph the probability distribution (density of probability) of the CPU time needed to find the optimal value (e.g., if in 100 trials the optimum is found after exactly 220 iterations, then for the value 220 of the abscissa we will have P(220) = 100/10,000). Fig. 7 shows that cooperation greatly improves the probability of finding quickly an optimal solution.